Measurement-First Behavioral Anomaly Detection at the Kernel Boundary: Reproducible Methods for eBPF-Based Network Security Evaluation

Naeem Roussas

Abstract—Detector claims are credible only when the methodology that produced them is credible. In extended Berkeley Packet Filter (eBPF)-based network security monitoring, simulation, userspace measurement, and kernel execution are easily conflated, and numbers from one tier are reported as if they came from another. We give a measurement-first evaluation pipeline that assigns every number an explicit provenance label: synthetic simulation, userspace measurement, kernel-path measurement, real capture, or namespace-lab. Every result is reproducible from committed seeds, configurations, and run artifacts. The primary result is a measured simulation-to-lab transfer: a behavioral anomaly detector iterated fully in simulation produced per-entity detection outcomes identical to those on real namespace traffic in 30/30 seeded runs, for a conformal-calibrated statistical detector and a rule-based comparator. The same discipline exposes divergence where it matters. At 16,000 flows/s the simulation predicts 12.3% flow-record loss for a ring-buffer logger, but the real kernel-path logger on virtual interfaces loses nothing in 2,097,775 flows: the ceiling is a simulation-model artifact, not a measured limit. Three roadmap assumptions are falsified with identified root causes. All kernel-path, real-capture, and namespace-lab evidence comes from virtual interfaces (WSL2 veth pairs), never a physical NIC or production traffic; the claims do not generalize beyond that scope.

Index Terms—Measurement methodology, eBPF, XDP, anomaly detection, count-min sketch, conformal prediction, graph-based intrusion detection, network namespaces, reproducibility

I. Introduction

Network security research has a measurement problem. Detector papers often report true-positive rates (TPR) computed on the same data used to tune the detector. They report false-positive rates (FPR) on benign models that do not contain the dynamics (DHCP rebinds, MAC randomization, VM migration) that cause false positives in production. They report throughput for "line-rate" telemetry pipelines whose record loss is never measured against ground truth. The networking community has started to correct this problem. CCEval [1] shows that even congestion-control evaluation, a mature subfield, systematically misestimates the metrics it reports. CCEval argues for evaluation frameworks that state their confidence and their failure modes explicitly. Closer to our setting, CALIBURN [2] applies conformal risk control to streaming network intrusion detection. CALIBURN derives alert thresholds from operator-committed false-positive budgets, not from post-hoc tuning. Empirical security has its own tradition of methodological rigor. Fuzzing research treats seeded, deterministic, differentially tested evaluation harnesses as a requirement for empirical claims [3], [4]. Symbolic-execution work made proof-against-oracle validation a standard practice [5].

This paper applies that culture to the evaluation of eBPF-based network security monitoring. The extended Berkeley Packet Filter (eBPF) and the eXpress Data Path (XDP) [7] make it cheap to deploy custom kernel telemetry. A growing ecosystem uses them for runtime security monitoring [8], [9]. But the kernel boundary is exactly where evaluation becomes sloppy. Ground truth is hard to construct in the kernel. Lab setups are hard to reproduce. It is tempting to report simulation numbers as if they were measurements. Our position is the following. A detector whose evaluation pipeline labels every number with its provenance is more valuable than a detector with better-looking numbers that cannot be verified. The provenance labels are simulation, measurement, or absent, with an executable placeholder.

We show this stance in a four-component pipeline. The pipeline mirrors a real eBPF monitoring stack, the parent research harness it comes from. A six-project namespace-lab campaign in that harness executes the same classes of detector logic against real packets on veth pairs. Each component was first validated in seeded simulation and userspace measurement. The lab-dependent components (P1, P3, P4) were then executed in a virtual-interface lab: native XDP on veth pairs and Linux bridges under Windows Subsystem for Linux 2 (WSL2), plus real tcpdump captures. We report the simulation arms together with those counterparts, the agreements and the divergences:

1) A provenance taxonomy and a reproducible methodology (Section IV). Every number carries an explicit execution category: SYNTHETIC SIMULATION, PIPELINE SIMULATION, USERSPACE MEASUREMENT, KERNEL-PATH MEASUREMENT, REAL CAPTURE, or NAMESPACE-LAB. Every experiment is seeded, deterministic, and re-runnable by a single command. Results carry confidence intervals (Wilson, bootstrap) and conformal calibration, not point estimates.

Naeem Roussas is with Southern New Hampshire University, Manchester, NH, USA (e-mail: naeem.roussas@snhu.edu). ORCID: 0009-0002-6190-0224 (https://orcid.org/0009-0002-6190-0224).

The companion source tree is included as supplementary material with this submission; all artifacts are re-runnable from committed seeds, configurations, and run artifacts.

2) A multi-component evaluation pipeline with oracles (Sections III–V): a behavioral anomaly detector with split-conformal threshold calibration, kernel-style data structures with proofs and empirical oracles, a graph-theoretic Layer-2 rogue-device detector with a formal account of when it must fire, and a flow-logger measurement harness with an exact accounting identity. Each component is checked against an independent oracle. The work includes a calibration bug that only out-of-sample calibration revealed.
3) Cross-tier evidence: a measured transfer and its divergences (Sections V and V-G). A simulation-iterated behavioral detector gave per-entity detection outcomes identical to real-namespace-traffic outcomes in 30/30 seeded runs. The kernel-path and real-capture arms confirm some simulation results and overturn others, including the flow-logger consumer ceiling (a simulation-model artifact) and the beacon and rogue-bridge classes under real jitter and the real Spanning Tree Protocol (STP).
4) A namespace-lab campaign over six detector/telemetry classes (Sections V-G–V-L): behavioral-detector transfer, DNS forgery, TCP timestamp/TTL anomalies, covert channels, flow-logger completeness, and XDP overhead, executed as real packets on virtual interfaces.
5) Explicit negative results and scoped limitations (Sections V-M–VI): three falsified roadmap assumptions with identified root causes, further negative results reported in place, and limitations stated as aggressively as the results.

All numbers in this paper trace to committed output files with recorded seeds, commands, and dates (Section VIII). Kernel-path and real-capture numbers are restricted to virtual interfaces on an owned WSL2 host. No number involves a physical NIC or an operational network. We say so wherever the distinction matters.

Notation. P1–P4 denote this repository's userspace prototypes (behavioral detector, kernel data structures, graph L2 analysis, flow-logger harness). NS1, NS2, NS3, NS4, NS5, NS7 denote the parent harness's namespace-lab roadmap projects p1-xdp-baseline, p2-flow-logger, p3-ttl-ts-anomaly, p4-dns-forgery, p5-covert-sketch, and p7-behavioral-baseline (numbering follows the roadmap; no NS6 exists in this set).

II. Background and Related Work (Summary)

Five threads of prior work frame this paper: evaluation-methodology critiques (CCEval [1], CALIBURN [2], and the fuzzing and symbolic-execution traditions [3]–[5]); eBPF/XDP security monitoring [7]–[10], [13]; behavioral detection, calibration, and flow ground truth [6], [11], [12], [14], [15]; sketching and streaming telemetry [16], [17], [20], [21], [24]–[28]; telemetry completeness and provenance [29]–[33]; and graph-based L2 detection [18], [19]. The full discussion of each thread, and our position against it, is given in the Appendix.

III. System Design

The four components form one coherent pipeline. The kernel boundary produces observations (P4's logger, P2's structures). Userspace scores them (P1's detector). A structural model catches what statistics miss (P3's graph detector). Each component is developed and evaluated in userspace, so the logic is fully validated before the kernel port. The kernel data path exists in the parent research harness and is out of scope for the numbers reported here. The namespace-lab projects then re-execute the same classes of logic over real packets. NS7 re-runs the P1 detector family, NS2 the P4 logger, and NS5 the P2 sketches. NS3/NS4 add protocol-level detectors with no userspace-prototype counterpart. Figure A.1 (Appendix) shows the data flow.

A. P1: Behavioral-baseline deviation detection

Per (entity, window), the detector computes four features over flow records: destination entropy, KL divergence of the window's destination distribution against the entity baseline, mean log-bytes, and the inter-arrival coefficient of variation. Features are normalized by robust z-scores (median/MAD) and combined by max. The threshold is not hand-tuned. It is the conformal quantile of the entity's own out-of-sample baseline scores (Section IV), targeting FPR $\alpha = 0.05$ [14], [15]. The feature set is deliberately restricted to what an eBPF map can maintain in-kernel: counters, histograms, reservoir samples, the P2 structures. The scoring prototype therefore drops onto the kernel data path unchanged. Evaluation runs over a seeded, deterministic generator: 24 entities with Zipf-ish destination profiles, log-normal volumes, and exponential inter-arrivals. A quarter of the entities become anomalous at the baseline/eval boundary. The anomaly classes are scanner, exfil, and beacon, motivated by ATT&CK [6].

B. P2: Kernel data structures in C11

Three structures, header-only C11, fixed memory, no allocation after initialization: the form they must have behind a BPF map interface.

a) Count-min sketch: $w \times d$ 32-bit counters with a salted FNV-1a hash family. By the Cormode–Muthukrishnan theorem [16], $w = \lceil e/\varepsilon \rceil$ and $d = \lceil \ln(1/\delta) \rceil$ give estimate$(x) \geq f(x)$ always and $\Pr[\text{estimate}(x) \leq f(x) + \varepsilon N] \geq 1 - \delta$. The proof is one line of Markov per row plus independence across rows. Per-row excess X_r has $\mathbb{E}[X_r] \leq N/w = \varepsilon N/e$, so $\Pr[X_r \geq \varepsilon N] \leq 1/e$. The min over d rows then fails with probability $\leq (1/e)^d = \delta$. We verify the bound empirically: a unit test observes a violation rate comfortably under δ at $\varepsilon = 10^{-3}$.

b) LRU flow cache: Fixed-capacity hash map plus doubly-linked list. Lookup, insert, and evict-least-recently-used are $O(1)$. This is the userspace analog of BPF_MAP_TYPE_LRU_HASH, with pool allocation and deletion-safe chaining.

c) Reservoir sampler: Algorithm R [17]: a uniform k-sample of a stream of unknown length in one pass, $O(k)$ memory. Uniformity is an induction on stream length. Item $n{+}1$ enters with probability $k/(n{+}1)$. An occupant present with probability k/n survives with probability $n/(n{+}1)$. The result is $k/(n{+}1)$. We check empirical uniformity with a Kolmogorov–Smirnov (KS) test of sampled stream positions against Uniform$[0, N)$ at $\alpha = 0.01$.

C. P3: Graph-theoretic L2 rogue-device detection

The observation graph $G_t = (M_t \cup P_t, E_t)$ is bipartite over MACs and switch ports, with an edge per observed binding in window t. The formal model has five definitions, two lemmas, three theorems, and a corollary, proved in the repository's docs/PROOF.md. It states:

- Benign model $\mathcal{B}$: every port binds ≤ 1 MAC (B1), every MAC binds ≤ 1 port (B2), churn is balanced—joins $\leq$ leaves per window (B3).
- Lemma 1: under $\mathcal{B}$, G_t is a matching.
- Theorem 1 (DEG): a rogue tap or rogue bridge, when observable, forces some port vertex to degree ≥ 2.
- Theorem 2: in a forest, v is an articulation point iff $\deg(v) \geq 2$; hence the AP signal (Tarjan [18], $O(V{+}E)$) fires exactly when DEG fires in this observation model.
- Theorem 3 (VERT): a rogue host occupying a port makes joins exceed leaves, which Lemma 2 rules out benignly.
- Corollary: with per-window observability probability q, detection latency in windows is geometric, $\Pr[L = k] = (1-q)^k q$.

The detector implements Tarjan's algorithm from scratch (stdlib-only). The test suite checks it against a brute-force articulation-point oracle on 50 random graphs. It also checks the Lemma-1 invariant empirically, and it checks determinism.

D. P4: Flow-logger measurement harness

The P4 arm is a faithful userspace model of the kernel logger data path. The model has three parts. The first is hash-map flow aggregation (≈ BPF_MAP_TYPE_LRU_HASH). The second is a fixed-size ring buffer (≈ bpf_ringbuf_reserve: push fails when full and the record is dropped and counted). The third is a batched consumer (100 Hz poll, 128 records/poll = 12,800 records/s service rate). A seeded synthetic trace (Poisson flow arrivals, Pareto packet counts, exponential durations) drives the model. The trace also emits the exact ground-truth flow records a perfect logger would produce. The sweep crosses configured offered load $\{0.5, 2, 8, 16\}$k flows/s with ring capacity $\{64, 256, 1024\}$. Configured offered load is the Poisson arrival-rate parameter of the generator, not an observed rate. The sweep measures completeness (emitted / ground truth), byte coverage, and ring drops. The consumer drains in batches per poll. At end of run a single final batch is drained. Records pushed but never drained (residual) can therefore remain in the ring. The accounting identity emitted + dropped + residual = ground truth is checked for every configuration. Zeek [11] is the conceptual ground-truth analog. In the lab validation, pcap-derived records play the role the synthetic ground truth plays here.

E. The namespace lab (NS1–NS7)

The namespace lab sits between simulation and a hardware testbed. Real packets traverse the real kernel TCP/UDP/ICMP stack over veth pairs between network namespaces created with unshare -rmn (no host root required). Kernel SO_TIMESTAMPNS timestamps are used where timing matters. The host is a WSL2 machine (kernel 6.18.33.1-microsoft-standard, 24 vCPU). Two constraints shape the results. First, the host sets kernel.unprivileged_bpf_disabled=2, so bpf() returns EPERM for rootless projects. NS2's reference eBPF flow logger compiles to a valid eBPF executable and linkable format (ELF) object but was not executed. The measured logger is its userspace analogue (AF_PACKET PACKET_RX_RING + dict flow table). NS1's XDP counter, by contrast, was loaded with privilege in native, JITed mode. Second, sibling roadmap projects ran concurrently on the host during all campaigns. Timing-sensitive intervals include that noise. Each project's RESULTS.md records per-run load averages, and nothing was excluded.

IV. Methodology

This section is the paper's reason to exist.

A. Seeds and determinism

Every experiment is driven by an explicit seed (P1: seeds 1000–1029; P2: seed 2024; P3: seeds 5000+ with benign runs at 15000+; P4: seed 4242; NS campaigns: per-project seed schedules recorded in their RESULTS.md files). The generators are unit-tested for byte-identical output under a fixed seed. Every figure in Section V is reproduced by one committed command: python figures/make_figures.py, which reads only the committed result files. The repository's results/RESULTS.md maps each output file to its producing script, seed, date, and provenance label. Results were produced on a single machine (13th Gen Intel Core i7-13700KF, Linux, Python 3.14.4, gcc 15.2.0, -O2). The simulation and userspace arms ran on 2026-08-18. The kernel-path and real-capture arms ran on 2026-08-29 on the same host (WSL2, kernel 6.18.33.1-microsoft-standard-WSL2). The namespace-lab campaigns ran on 2026-08-20 on the WSL2 namespace host of Section III-E. Kernel-path runs additionally record the full loader toolchain, cilium/ebpf v0.15.0, and bpftool/BTF provenance per run manifest.

B. Execution categories: provenance labels on every number

Each committed number carries an explicit execution category. Table I defines the six categories once; the same

TABLE I
Execution categories (provenance labels) assigned to every reported number. NS4's FPR-scaling and NS5's area under the ROC curve (AUC) sweep studies are Monte Carlo and are labeled SYNTHETIC SIMULATION where they appear.

Category	Definition (arms)
SYNTHETIC SIMULATION	real analysis code over seeded synthetic input, userspace only (P1, P3 simulation arms)
PIPELINE SIMULATION	userspace model of the kernel ring-buffer data path driven by a seeded synthetic trace (P4 simulation arm)
USERSPACE MEASUREMENT	the code under test ran on this machine, and the numbers describe that run (P2 C11 benchmark; explicitly not eBPF maps and not a kernel path)
KERNEL-PATH MEASUREMENT	real compile-once–run-everywhere (CORE) eBPF XDP program attached in native-driver mode to a veth pair in private network namespaces on an owned WSL2 host (P4 kernel-path arm, Section V-E)
REAL CAPTURE	unmodified simulation detector code fed with records from real tcpdump captures of scripted workloads on netns/veth/bridge topologies (P1, P3 real-capture arms, Section V-F)
NAMESPACE-LAB	real packets through the real kernel stack on veth pairs between network namespaces (the NS headline numbers, Sections V-G–V-L)

labels appear on every table and figure in Section V. Namespace-lab is not hardware-lab. There is no NIC, no driver queues, and (for the rootless projects) no in-kernel eBPF. We do not let it impersonate one. NS1's XDP counter is the exception, loaded with privilege in native JITed mode. The last three categories are virtual-interface results only. No number in this paper involves a physical NIC, a physical switch, or an operational network. Nothing here generalizes to those settings. Where a category boundary still binds (physical-NIC rates, production benign dynamics), the gap is stated as a placeholder, not estimated (Section VII).

C. Confidence intervals, not point estimates

All proportions report Wilson 95% confidence intervals (CIs) [22]. Mean time-to-detect (TTD) reports a seeded bootstrap 95% interval [23] (percentile method, seed 0, $B = 2000$ resamples, over the 30 run-level mean TTDs, not pooled entity TTDs). Small-n perfect scores are reported with their intervals. P3's TPR $= 10/10$ is $[0.722, 1.0]$, not "100%."

D. Split-conformal calibration—and the bug it caught

P1's per-entity threshold is the conformal quantile of baseline window scores with the finite-sample correction $\lceil (n+1)(1-\alpha) \rceil / n$ [14], [15]. Two design decisions make the FPR claim honest. Both exist because naive versions failed during development. First, an interleaved split: even-indexed baseline windows fit the median/MAD normalization, and odd-indexed windows calibrate the threshold. Calibration windows are therefore scored against a model they did not contribute to, exactly the position of eval windows. Second, leave-one-out KL: the first implementation scored each fit window's KL feature against a reference distribution that included the window itself. KL divergence against an in-sample reference is biased downward. Feature medians were therefore optimistic, and thresholds were mis-calibrated. The fix scores each window against the leave-one-out reference (fit_counts $- c$), which is how eval windows are necessarily scored. The lesson generalizes. Any detector statistic computed against a reference the sample contributed to is a leak. Only out-of-sample calibration reveals it.

E. Differential testing and oracle verification

Following the fuzzing tradition [3], [4], every structure and detector is checked against an independent oracle: a naive Python mirror of the P2 structures (byte-identical traces on three seeds), a brute-force articulation-point oracle for Tarjan (50 random graphs), statistical oracles for the CMS bound and reservoir uniformity, and an independent tcpdump-to-pcap parse for NS2. The full protocol is given in the Appendix. The full suite (25 tests) passes on the committed tree.

F. Experiment configuration

Table A.1 fixes every degree of freedom of the evaluation, so the reader does not have to trust our defaults. It lists run counts, seeds, generator parameters, and the single committed command that reproduces each result file. The generators are deliberately simple and fully specified. We prefer that the reader criticize an explicit model than wonder about an implicit one.

V. Evaluation

Every table below is labeled with its execution category (Table I). The kernel-path and real-capture subsections (V-E, V-F) describe executed virtual-interface lab runs, never extrapolated to physical NICs. Table II maps each headline number to its source and label. The per-project subsections unpack them; the detailed per-component tables and figures are collected in the Appendix.

A. P1: Behavioral detector (SYNTHETIC SIMULATION)

Table A.4 reports entity-level TPR, window-level FPR, and time-to-detect over 30 seeded runs (seeds 1000–1029; 180 anomalous entities, 10,796 benign windows). The headline finding is methodological. The conformal-calibrated detector's empirical FPR (0.029) respects its 0.05 target with room to spare. The interval $[0.026, 0.033]$ is too tight for this to be luck. The TPR of 0.761 is a real detection limit of the feature set, not a calibration failure. Section VI discusses which anomaly classes are missed. NS7 (Section V-G) re-runs this detector family against real namespace traffic.

TABLE II
Headline results and provenance map. SIM = synthetic/pipeline simulation, MEAS = userspace measurement on the development machine, KP = kernel-path measurement (native XDP on veth, WSL2), RC = real capture (WSL2 virtual interfaces), NS-LAB = NAMESPACE-LAB: real packets on veth pairs, no NIC, no privileged eBPF unless noted. Units: pps = packets per second, kpps = thousands of packets per second, Mbps = megabits per second.

ID	Key result	Prov.
P1	TPR 0.761 [0.694, 0.818]; FPR 0.029 vs. 0.05 conformal target	SIM
P2	CMS heavy-hitter rel. err 3.36 → 0.002 (1 KB → 256 KB) at 33–37 M upd/s	MEAS
P3	TPR 1.0 for 3 rogue classes; FPR = 0 by model construction	SIM
P4	completeness 1.0 ≤ 2k flows/s; caps at 0.877 above consumer capacity	SIM
P4K	completeness 1.0 (2,097,775 flows), incl. 16k flows/s; 16k sim divergence is an artifact	KP
P1R, P3R	real-capture TPR drops 0.761 → 0.556, beacon 0.717 → 0.208; rogue bridge 10/10 → 7/10 under STP	RC
NS7	detection outcomes identical sim vs. lab in 30/30 runs, both detectors	SIM+ NS-LAB
NS4	DNS forgery TPR 1.000 (3058/3058), FPR 0/14,917, ∼100 μs alerts	NS-LAB
NS3	TTL classes AUC 1.000; timestamp drift AUC 0.9905	NS-LAB
NS5	sketch/exact AUC parity ≤ 2.3 pts; MQTT transfer 0.917–1.000	SIM+ NS-LAB
NS1	XDP counter +11.0% CPU/pkt at 64 B; tc mirror cheaper (+6.8%) on veth	NS-LAB
NS2	flow logger lossless ≤ 615 Mbps; knee is pps-bound (∼300 kpps)	NS-LAB

B. P2: Kernel data structures (USERSPACE MEASUREMENT, this machine)

Table A.5 shows the count-min sketch. Heavy-hitter relative error falls from 3.36 to 0.002 as memory grows from 1 KB to 256 KB, at a steady ∼33–37 M updates/s (single-threaded, gcc -O2). This is consistent with the $\varepsilon \propto 1/w$ bound. On this trace it is in fact faster than $1/w$ (observed factors of ≈5–8 per 4× of width). The same table shows the LRU cache on a Zipf(1.1) trace. The hit rate grows 0.244 → 0.778 as capacity grows 16 → 4096 entries, at ∼50–61 M ops/s. The reservoir sampler sustains 321.8–323.8 M offers/s in the committed run of 2026-08-18 (p2_benchmarks.csv). All P2 tables in this paper report that run. Throughput varies run to run. A re-measurement on the same machine on 2026-08-29 gave 335.3–335.6 M offers/s (p2_benchmarks.rerun.csv). Across repeated runs we observed 305–336 M. The KS uniformity test passes at $\alpha = 0.01$ for both $k \in \{64, 1024\}$. Each configuration is timed once (single-shot, CLOCK_MONOTONIC, no warm-up, no repetitions). Throughput figures are therefore live benchmarks that carry no interval. Error and uniformity figures at seed 2024 are seed-deterministic.

C. P3: L2 rogue-device detection (SYNTHETIC SIMULATION)

Table A.6 shows perfect detection of all three rogue classes, at n=10 per class and therefore with wide small-n intervals ($[0.722, 1.0]$, Wilson). We report the latency distributions as measured, without claiming model fit. Rogue tap (mean 3.0 s) and rogue bridge (15.0 s) are consistent with their $q = 0.7$ geometric observability models. The rogue-host mean (27.0 s) deviates upward from its own $q = 0.8$ model ($P \approx 0.004$ under that model at n=10). This is a deterministic small-sample effect of the committed seeds; we report it rather than smooth over it. The benign FPR is 0/450 windows, and we insist the reader not be impressed. Under the benign model $\mathcal{B}$ the DEG/VERT/AP signals cannot fire: Theorems 1 and 3 are proven no-benign-firing results. Zero is therefore the only possible outcome of this experiment. What the simulation actually validates is (i) implementation correctness (the detector fires exactly when the theorems say it must) and (ii) latency distributions. The Wilson upper bound on benign FPR is 0.0085. The operationally meaningful non-zero FPR question requires benign dynamics that violate B1–B3 (MAC randomization, DHCP rebinds, VM migration) and is deferred to the lab (placeholder, Section VII). The benign denominator was initially committed only as an aggregate in p3_summary.json (30 runs × 15 windows = 450). The artifact now commits the raw per-run benign records (p3_benign_per_run.csv, regenerated bit-for-bit from seeds 15000–15029 by p3_graph_l2_analysis/src/dump_benign_runs.py). The 0/450 figure is therefore auditable from committed raw records as well as seed-reproducible.

D. P4: Flow-logger completeness (PIPELINE SIMULATION)

Table A.7 gives the completeness sweep. Three regimes are visible. Below consumer capacity (500–2,000 flows/s configured offered load): completeness is exactly 1.0 at every ring size. The logger is lossless and buffer size is irrelevant. Near saturation (8,000 flows/s, burst pressure above instantaneous service): a ring of 64 loses 4,316 of 33,148 records (completeness 0.870) while ring ≥ 256 remains lossless. Buffering absorbs bursts, as queueing intuition predicts. Above consumer capacity (16,000 flows/s configured offered load; realized ended-flow rate 13,285/s > 12,800 records/s consumer): completeness caps at 0.877 for ring ≥ 256 regardless of buffer size. No finite buffer fixes an undersized consumer; it only delays the drop. The undersized ring additionally collapses to 0.467. Byte coverage tells a subtler story. It sits at 0.78–0.80 even where record completeness is exactly 1.0, because its denominator is all ground-truth bytes, including heavy-tailed flows still in flight at the end of the 5 s window. In other words, the two metrics have different denominators by construction. Completeness answers "did the logger emit a record for every finished flow." Byte coverage answers "what fraction of observed bytes is described by emitted records so far." Quoting either alone would mislead. Under overload the two diverge further (0.681 vs. 0.877 at the 16,000 flows/s setting, ring 256), because dropped records come from the same heavy-tailed size distribution that dominates bytes. A logger benchmarked

below its consumer ceiling will report 100% completeness forever. The ceiling is a consumer property, not a buffer property. NS2 confirms the same shape live (Section V-L): the knee is set by per-packet consumer cost, and loss is counted, not silent.

E. P4 kernel path: a real CO-RE/ringbuf logger (KERNEL-PATH MEASUREMENT, native XDP on veth, WSL2 virtual interface)

The userspace model above makes a strong prediction: record loss occurs once the realized record rate exceeds the 12,800 records/s consumer. We built the real thing to check it (p4k_kernel_flowlogger/). A CO-RE XDP program (bpf/flowlog.c) aggregates packets into a hash flow table. On flow end, it emits a 120-byte record via bpf_ringbuf_reserve. Reserve failure increments a kernel drop counter, the same loss site the simulation models. A Go loader/consumer attaches in native-driver XDP mode on a veth pair in private network namespaces (owned WSL2 host; never a physical NIC). A seeded generator (Poisson flow arrivals, 90 % UDP / 10 % real TCP) offers the load. The generator manifest and an independent tcpdump pcap serve as dual ground truth. The arm has 54 gated runs. The base matrix crosses 4k/8k/16k flows/s with ring 256/1024/4096 records over 5 seeded reps (45 runs). Nine declared slow-consumer runs (100 μs/record consumer delay at 16k flows/s) exercise the ring-full loss path.

a) Kernel vs. simulation: agreement, then an informative divergence: Table A.8 (and Figure A.3) is the main result of this comparison. At 8,000 flows/s the arms agree exactly (completeness 1.0 both). At 16,000 flows/s the simulation predicts 0.877 while the kernel path measures 1.0. Neither arm is buggy. The sim's consumer is a 100 Hz $\times$ 128-batch poller capped at 12,800 records/s. At 16,000 flows/s, offered load exceeds its drain rate regardless of ring size (the sim loses the same 8,198 records at ring 256 and 1024). The real epoll-driven ringbuf reader sustains $\geq$16k records/s even with the smallest ring. The simulation correctly identifies the mechanism, consumer drain rate $\times$ ring capacity, as the slow-consumer runs confirm. But its specific consumer parameters understate a real Go ringbuf consumer. This is a simulation fidelity finding. It is the strongest argument in the paper for the provenance-label discipline: without a kernel-path arm, the 0.877 "ceiling" would have read as a measurement.

F. P1/P3 real-capture arms (REAL CAPTURE, WSL2 virtual interfaces)

Two complementary arms feed the unmodified simulation detector code with records derived from real captures on virtual interfaces (netns, veth pairs, Linux bridges; tcpdump pcap ground truth; no external network contact). Windows are shorter than in simulation for wall-clock reasons (P1: 15 s vs. 30 s; P3: 10 s vs. 30 s), so time-to-detect is compared in windows, and TPR/FPR directly.

a) P1 real capture (summary): Pooled TPR falls from 0.761 to 0.556 [0.441, 0.665], driven by the beacon class (0.717 $\rightarrow$ 0.208) under real scheduling jitter; calibration holds (lab FPR 0.0461 [0.040, 0.053] vs. the 0.05 target).

b) P3 real capture (summary): Rogue host and rogue tap replicate perfectly (10/10). Rogue bridge drops to 7/10 [0.397, 0.892]: real 802.1d STP suppresses the rogue link in 3/10 runs. Benign FPR stays 0/150 windows (Wilson upper bound 0.025). The full per-arm detail (Tables A.2 and A.3, Figure A.2) is given in the Appendix.

G. NS7: Simulation-to-lab transfer of the behavioral detector (NAMESPACE-LAB)

This is the paper's headline result. NS7 re-implements the P1 detector family, extended with diurnal buckets and a rate feature. It runs in two modes over the same seeded workload: simulation (the planned schedule used directly as the flow log, zero jitter/loss) and namespace-lab (real TCP/UDP traffic over a veth pair, flow log measured server-side). Time is scaled 360$\times$: a 5.0 s real window is a 30-minute simulated window, with a 24 h baseline and a 6 h eval. Across 30 seeded runs ($\sim$598k real flows, 30/30 runs completed with zero client errors and mean loss 0.0000), per-entity detection outcomes were identical in all 30 runs for both detectors: a conformal-calibrated statistical detector and a rule-based comparator (Table A.10). Only 4 of 60 run-detector pairs differ at all, and only in benign-window FP counts ($\pm$1–3 windows). In one run, real timing jitter pushed a single flow across a window boundary. At this traffic volume (mean $\sim$70 flows/s) and sub-ms jitter, flow-record-level behavioral detection is insensitive to whether records come from a synthetic generator or real packets. The simulation prototype's detection results transfer to the namespace lab unchanged. This validates iterating detectors in cheap simulation, with namespace runs as confirmation.

The per-class breakdown shows the detectors are complementary, identically in both modes. Rules win on novel-dest (TPR 1.000 vs. 0.600): the never-seen-destination rule fires on a single flow, while the conformal threshold is too conservative in sparse windows. Stats win on beacon (0.933 vs. 0.533): the periodicity rule needs $\geq$ 8 same-destination flows in one window, which quiet-hour windows never accumulate. Both catch exfil (1.000) and off-schedule ($\geq$ 0.933). The measured FPR (0.077–0.105 per 30-min window) is higher than the P1 prototype's 0.029, for a traced reason. NS7 calibrates on only 6 windows per (entity, 6h-bucket), where P1 uses 20 per entity. This defeats the roadmap's "$<$ 1% daily FPR" hypothesis (Section V-M). Baseline stationarity checks (KL between baseline halves; Spearman trend of benign scores) show no drift in either mode.

H. NS4: DNS forgery detection (NAMESPACE-LAB)

NS4 races a forger against an in-namespace resolver over a veth pair. The detector (a userspace DnsGuard) keys in-flight queries by (TXID, qname, source port) and alerts

on duplicate or conflicting responses—the QUANTUM-class signal [13]. The workload is Poisson query streams of 10–250 queries per second (qps) (12 s runs, 3 reps/rate). Every one of 3,058 injected forgeries was detected: pooled TPR 1.000, $[0.9987, 1.000]$. There were zero false positives in 14,917 queries (FPR 0, $[0, 2.6 \times 10^{-4}]$). Median alert latency is $\sim$100 μs (processing 83.5–108.8 μs; end-to-end 94.1–116.2 μs). Latency grows only $\sim$25% from 10 to 250 qps. This is an order of magnitude below the observed $\sim$1 ms race window, so a pre-cache enforcement point has ample time. Detection is independent of who wins the race: the 1.05% of forgeries that arrived first were still caught when the legit response arrived second. Two boundary results matter more than the perfect score. First, DNS over TLS (DoT) blindness: 72 forged answers injected as TCP/853 frames with opaque payloads produced 0 alerts (TPR 0.000, $[0, 0.050]$). Port-based L3/L4 DNS inspection is fundamentally blind to encrypted DNS transports. Second, the keying choice is critical (Section V-M). A Monte Carlo FPR study (SYNTHETIC SIMULATION) at 100k qps finds full (txid, qname, sport) keying at 6.9×10^{-5} (1 ms round-trip time (RTT)) to 3.0×10^{-3} (50 ms RTT) false positives per query. TXID-only keying fails at WAN RTT by a birthday effect (Section V-M).

I. NS3: TCP timestamp/TTL anomaly detection (NAMESPACE-LAB)

NS3 detects middlebox-rewritten and crafted flows from passive TTL and TCP-timestamp features (5 seeds; 4,000 benign + 1,000 per-class flows pooled; live capture fidelity 99.94–99.96%). Table A.9: a real iptables TTL-set middlebox rewrite and per-packet random TTLs are perfectly separated from benign (AUC 1.000, TPR 1.000 at 1% and 0.1% FPR). Non-monotonic timestamps are separated as well. The timestamp-drift class is the only non-saturated one (AUC 0.9905 $[0.9877, 0.9930]$, TPR 0.945 at 1% FPR). The cause is measured, not guessed. Under host CPU contention the sender's pacing genuinely bursts on the wire, and a passive observer cannot tell a fast clock from burst arrival. The simulation (modeled path, same seeds) scores all four classes at AUC 1.000, so the entire sim→live delta is live timing noise. A companion run gives an honest negative result. Replacing userspace recv() timestamps with kernel SO_TIMESTAMPNS timestamps did not help (AUC 0.9911 vs. 0.9905). The noise is on the send path, not the capture path. Per-class detectors also beat a fused max-score: the combined detector inherits the noisy benign TS tail. This argues for independent per-feature thresholds.

J. NS5: Covert-channel detection with sketches (SIMULATION + NAMESPACE-LAB)

NS5 asks whether the P2 sketch machinery is good enough to detect covert channels. A streaming detector with fixed $\sim$1.8 KB per-flow memory (count-min tables) is compared against an exact offline baseline that stores every event. The study sweeps seeded timing (pulse-position) and size (payload-parity) channels at 0.0125–3.2 bps (SYNTHETIC SIMULATION, 120 test windows per cell). The sketch is within $\pm$2.3 AUC points of the exact baseline at every rate, for both channel kinds. Both detectors saturate at AUC $\approx$ 1.0 at $\geq$ 0.4 bps. The evasion floor is quantified. Both detectors are statistically blind (AUC CI includes 0.5) at $\leq$ 0.025–0.05 bps with a 120 s window: a sender slower than $\sim$1 bit per 40 s is invisible at these sample sizes. Both detectors are reliable (CI low $\geq$ 0.9) at $\geq$ 0.4 bps. The transfer test (NAMESPACE-LAB) trains on synthetic windows and applies the models unchanged to real MQTT 3.1.1 over kernel TCP on a veth pair. Sketch AUC is 0.917–1.000, where the exact baseline scores 0.792–1.000. The sketch ordering survives real-network jitter. One sub-claim honestly fails. The roadmap's "$\sim$100$\times$ lower latency" is not confirmed. Both detectors are window-bound at 120 s. The real streaming advantage is fixed per-flow memory that fits in a BPF map. This enables the same detection inline, instead of post-hoc on pcaps.

K. NS1: XDP telemetry overhead (NAMESPACE-LAB; privileged native XDP)

NS1 measures a minimal XDP counter (23 instructions: bpf_ktime_get_ns() + per-CPU map update, XDP_PASS) attached in native JITed mode to a veth endpoint. The comparison arms are a tc mirred mirror and an uninstrumented baseline (30 interleaved seeded reps per condition). The instrument is validated first: the XDP map's packet count matches the userspace receiver within 0.002% across all 121 XDP reps. At 64 B packets, 200k pps, loss-free, the counter costs +10.95% CPU per delivered packet $[+10.37, +11.53]$ ($\approx +0.64$ μs/pkt on a 5.82 μs/pkt path). The tc mirror costs +6.77% $[+6.20, +7.35]$. Tc is cheaper than XDP, which inverts the common assumption, and the hypothesized >10% tc penalty never appears. At 1400 B the same program is -0.46% (n.s.): overhead is a per-packet cost, visible exactly in the small-packet regime. Tail latency reacts long before medians (+19.9% p99 RTT at 64 B). Unpaced max-rate numbers (e.g., -47.3% delivered pps) are dominated by a veth artifact: attaching XDP switches the peer to an xdp_frame bulk path, and the receiver drops the resulting bursts. NS1 flags these numbers as artifacts rather than reporting them as XDP cost. The 10-gigabits-per-second (Gbps) leg of the hypothesis is untestable here: veth reaches at most $\sim$1.9 Mpps, 8$\times$ below the 14.88 Mpps operating point.

L. NS2: Flow-logger completeness, live (NAMESPACE-LAB)

NS2 is the namespace-lab counterpart of P4. Its flow logger has an AF_PACKET 64 MiB ring plus a dict flow table. (The compile-verified eBPF/ringbuf reference could not run rootless, Section III-E.) A seeded traffic generator drives the logger. The generator's exact ground-truth accounting is cross-validated by an independent tcpdump/pcap parse (200,000/200,000 packets accounted, completeness 1.0). The loss curve has the simulated shape with a live knee. There is zero loss at every

rate up to and including 250 kpps (∼615 Mbps at 256 B payloads; exactly 0 drops in all 3 reps at every such rate). There is also 1.72 Gbps with zero drops at 200 kpps with 1024 B payloads. Loss is packet-rate-bound, not byte-bound. Between 250k and 300k pps the kernel ring outpaces the consumer and loss begins. The loss is counted via tp_drops, not silent. Flow completeness degrades far more slowly than packet completeness (0.9969 vs. 0.9450 at 300 kpps): most flows are still detected, but per-flow accounting shrinks. Precision is exactly 1.0 everywhere: the logger never invented a flow. The hypothesis "ring-buffer flow logging sustains 1 Gbps with $< 0.1\%$ event loss" is supported for the userspace analogue. The perf-buffer comparison is untested (no eBPF variant runnable).

M. Falsified assumptions

Three roadmap hypotheses were rejected with identified root causes; we consider these as valuable as the confirmations, because each removes an assumption a reader might otherwise import.

a) XDP is cheaper than tc mirroring (NS1): On veth, the ordering inverts: the XDP counter costs +11.0% CPU per 64 B packet where tc mirred costs +6.8%. Root cause: the veth xdp_frame bulk path flips the skb-allocation economics that make XDP cheap on real drivers. The per-packet cost itself ($+0.64\,\mu$s) is real, but the comparison does not transfer. The 10 Gbps operating point is untestable on veth (∼1.9 Mpps ceiling vs. 14.88 Mpps required).

b) TXID-only keying suffices for in-flight DNS correlation (NS4): Falsified at WAN RTT: TXID-only keying hits 7.1% FPR at 100k qps/50 ms RTT, where full (txid, qname, sport) keying stays at 3.0×10^{-3}. Root cause: TXID reuse among in-flight queries is a birthday effect. FPR $\approx N/65536$ with $N = \text{rate} \times \text{RTT}$ (N=5000 predicts 7.6×10^{-2}; measured 7.1×10^{-2}).

c) 24h baselining yields $< 1\%$ daily FPR (NS7): Not confirmed: with 48 windows/day, a per-window FPR of 0.077–0.105 compounds to P($\geq$ 1 FP/entity/day) $\approx$ 0.99. Root cause: only 6 calibration windows per (entity, bucket). The finite-sample conformal correction therefore forces the threshold to the max calibration score, and the achieved window FPR (7.7–10.5%) exceeds the 5% target. Reaching $< 1\%$ daily FPR needs per-day alert budgeting (e.g., window-run persistence), not per-window calibration.

Two further negative results are reported in place. Kernel timestamps do not rescue timestamp-drift detection (Section V-I). The sketch detector's latency advantage is window-bound, not 100× (Section V-J).

VI. Limitations and Threats to Validity

We state these aggressively. The paper's thesis is that others must do the same.

Simulation is not the kernel—and we measured the gap. The P1, P3, and P4 simulation numbers validate logic. The kernel-path (P4) and real-capture (P1, P3) arms measure how far that logic survives execution on virtual interfaces. The answer is mixed, and we report both directions. P4 completeness agrees at 8k flows/s and diverges at 16k (a sim consumer-parameter artifact, Section V-E). P1's pooled TPR and beacon class degrade on real captures. P3's rogue-bridge class degrades under real STP, while benign FPR stays zero. NS7 shows detection outcomes transferring at flow-record granularity; nothing there predicts privileged-kernel or NIC-rate performance. All of this remains virtual-interface evidence: no physical NIC, no physical switch, no production traffic. The generators are deliberately simple (Zipf-ish, log-normal, Poisson/Pareto), NS5's benign model is a two-state synthetic process, and single-flow captures leave multi-flow sketch contention untested. Real traffic has diurnal structure, middlebox weirdness, and adversaries who read papers.

Namespace-lab is not hardware. All NS numbers come from veth pairs on a WSL2 host (kernel 6.18.33.1-microsoft-standard): no NIC, no driver queues, no interrupt requests (IRQs), virtualized scheduling. Absolute throughput and timing figures are host-specific. NS1's max-rate regime is dominated by a veth artifact, and the tc-vs-XDP ordering is expected to flip on a real driver. The transfer claims we make are deliberately flow-record-level, not rate-level.

Unprivileged-BPF restriction. The host's kernel.unprivileged_bpf_disabled=2 blocks bpf() for rootless projects. NS2's loss thresholds therefore describe the userspace ring implementation, not eBPF ringbuf. The eBPF variants are compile-verified only. Only NS1 executed in-kernel eBPF (privileged, native JITed). In-kernel execution of the NS2/NS4 data paths remains unmeasured.

Sibling-CPU contention. Roadmap projects ran concurrently on the 24-vCPU host (load averages logged per run). Timing-sensitive intervals (NS1 CPU costs, NS3 timestamp features, NS2's knee variance) include that noise. Zero-loss regions were unaffected: exactly 0 drops in every rep at every sub-knee rate.

P3's zero FPR is a theorem, not a result. The benign model excludes exactly the phenomena that produce false positives. The real-capture arm improves on this: benign FPR stayed 0/150 windows (Wilson upper bound 0.025) under real 802.1d STP churn on Linux bridges. But 150 windows of scripted benign lab churn is a finite sample, not proof of zero FPR, and it is still not production benign traffic. MAC randomization, DHCP rebinds, and VM migration at scale remain unmeasured. The benign denominator itself was initially a summary-only aggregate; the raw per-run benign records are now committed (Section V). Likewise, a fully passive tap is invisible to the model by definition. Detecting it needs content-addressable-memory (CAM) table cross-correlation, which is out of scope.

Real-capture arms are smaller and shorter-windowed. The P1 real arm is 12 runs (vs. 30 simulated) at 15 s windows. P3 is 10 runs/cell at 10 s windows with a benign denominator of 150 windows (vs. 450). Intervals are

correspondingly wider and are reported. TTD comparisons are made in windows, not seconds. The P1 beacon collapse and the P3 rogue-bridge misses each rest on small n (5/24 and 3/10 events respectively). They are directionally clear and quantitatively preliminary.

P1's TPR is a feature-set limit. 0.761 means a quarter of injected anomalies evade entropy/KL/volume/timing features entirely (low-and-slow exfiltration below baseline variance). The real arm widens that to nearly half (0.556), concentrated in the beacon class. NS7's extended variant reaches 0.867/0.883 (stats/rules) in both modes, with misses concentrated in the novel-dest and beacon classes under quiet-hour sparse windows. We report it because a methods paper that only reports its wins teaches nothing. The sim TTD interval $[35.6, 72.1]$ s is wide (n=30). More runs would tighten it. We chose to report n=30 honestly rather than run until the interval looked good.

The in-sample bias we almost shipped. The KL-calibration bug of Section IV was caught only because the evaluation protocol included out-of-sample calibration. A detector evaluated in-sample would have reported a falsely comfortable FPR. We cannot prove that no similar bug survives. We can only show the protocol that caught this one.

P2 throughput variance. Benchmarks on a single shared machine vary run to run; Section V reports the committed run's numbers together with the observed re-run spread. Each configuration is timed once, so we report the committed run's numbers and flag the variance rather than selecting the best run. Error/uniformity figures are seed-deterministic and do not vary.

Remaining lab gaps. The executed arms leave six unproduced items (physical-NIC rates, production benign dynamics, in-kernel P1 features, privileged-kernel NS2/NS4 execution, Zeek ground-truth diffs, and a real day of benign traffic). They are enumerated in the Appendix and in Section VII.

VII. Future Work

The deferred measurements are, in dependency order: physical-NIC rates for P4 and NS1/NS2, kernel ports of the P2 structures, MAC-randomization robustness for P3, P1 on in-kernel feature maps, VM-lab eBPF execution of the NS2/NS4 data paths, multi-flow contention and sub-window detection for NS5, and stronger ground truth (Zeek conn.log diffs, evasion corpora). The full enumeration is given in the Appendix.

VIII. Reproducibility and Artifact Availability

The complete artifact tree contains source, seeded generators, test suites, committed result files, and per-project READMEs with exact run commands. The top-level results/RESULTS.md is the provenance index: it maps each result file to its producing script, seed, date, and provenance label. The companion source tree accompanies this submission as supplementary material; bulk raw capture data are excluded from the archive and are regenerated bit-for-bit from the committed seeds by the documented scripts.

IX. Conclusion

A security detector is a claim, and a claim needs an evaluation that could, in principle, embarrass it. We built such an evaluation for an eBPF-based monitoring stack. It is seeded and deterministic end to end. It reports intervals instead of point estimates. It uses conformal calibration that catches its own bugs and differential testing against independent oracles. It puts a provenance label on every number. The central empirical finding is that the discipline pays off: detection outcomes iterated in pure simulation transferred unchanged to real namespace traffic, for two different detectors. The components perform credibly within those labels. Conformal FPR control holds empirically in simulation and stayed calibrated on real captures. Sketch error tracks theory over two and a half decades of memory. Structural detection has proofs of exactly when it must fire. The flow-logger loss curve identifies the consumer, not the buffer, as the ceiling, and the namespace lab confirms that curve's shape live, with loss counted rather than silent. This revision then did to itself what it asks of others. It executed the kernel-path and real-capture arms and let them embarrass the simulation where they could. They did, in instructive places: a real ringbuf consumer refutes a consumer-ceiling prediction, real jitter dissolves a beacon signature, and real STP hides a rogue bridge. The methodological takeaway is narrow and, we believe, durable. Simulation can transfer in defined circumstances, but only execution can show when it does. A pipeline that labels every number with its provenance makes that check possible, and it keeps each claim inside the evidence that supports it.

Conflict of Interest Statement

The author declares no conflict of interest.

References

[1] T. Liu, K. Gao, L. Chen, D. Li, J. Guang, X. Chen, V. Liu, Z. Chen, Y. Zhang, N. Jin, and R. Zhang, "CCEval: Accurately and confidently estimating performance metrics of congestion control algorithms for datacenter networks," in Proc. 23rd USENIX Symp. Networked Systems Design and Implementation (NSDI), Renton, WA, USA, 2026, pp. 2175–2189.

[2] M. A. Youssef, "CALIBURN: Operationally calibrated streaming intrusion detection with regime-dependent conformal risk control," arXiv:2605.24696, 2026. [Online]. Available: https://arxiv.org/abs/2605.24696

[3] M. Zalewski, "American fuzzy lop," [Online]. Available: https://lcamtuf.coredump.cx/afl/

[4] A. Fioraldi, D. Maier, H. Eißfeldt, and M. Heuse, "AFL++: Combining incremental steps of fuzzing research," in Proc. 14th USENIX Workshop on Offensive Technologies (WOOT), 2020.

[5] R. Baldoni, E. Coppa, D. C. D'Elia, C. Demetrescu, and I. Finocchi, "A survey of symbolic execution techniques," ACM Computing Surveys, vol. 51, no. 3, pp. 50:1–50:39, 2018.

[6] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," MITRE, Tech. Rep., Mar. 2020. [Online]. Available: https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf

[7] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, "The eXpress data path: Fast programmable packet processing in the operating system kernel," in Proc. 14th Int. Conf. Emerging Networking Experiments and Technologies (CoNEXT), Heraklion, Greece, 2018, pp. 54–66.
[8] Aqua Security, "Tracee: Linux runtime security and forensics using eBPF," [Online]. Available: https://github.com/aquasecurity/tracee
[9] The Falco Authors, "Falco: Cloud-native runtime security," [Online]. Available: https://falco.org
[10] J. Patel, L. Graeff Buhl-Nielsen, A. Ghosn, and M. Kogias, "KRAKENGUARD: Towards fine-grained eBPF isolation," in Proc. 23rd USENIX Symp. Networked Systems Design and Implementation (NSDI), Renton, WA, USA, 2026, pp. 2685–2704.
[11] V. Paxson, "Bro: A system for detecting network intruders in real-time," Computer Networks, vol. 31, no. 23–24, pp. 2435–2463, 1999.
[12] The Zeek Project, "The Zeek network security monitor," [Online]. Available: https://zeek.org
[13] Fox-IT Security Research Team, "Deep dive into QUANTUM INSERT," Apr. 2015. [Online]. Available: https://blog.fox-it.com/2015/04/20/deep-dive-into-quantum-insert/; detection code and pcaps: https://github.com/fox-it/quantuminsert
[14] V. Vovk, A. Gammerman, and G. Shafer, Algorithmic Learning in a Random World. Springer, 2005.
[15] G. Shafer and V. Vovk, "A tutorial on conformal prediction," Journal of Machine Learning Research, vol. 9, pp. 371–421, 2008.
[16] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," Journal of Algorithms, vol. 55, no. 1, pp. 58–75, 2005.
[17] J. S. Vitter, "Random sampling with a reservoir," ACM Transactions on Mathematical Software, vol. 11, no. 1, pp. 37–57, 1985.
[18] R. E. Tarjan, "Depth-first search and linear graph algorithms," SIAM Journal on Computing, vol. 1, no. 2, pp. 146–160, 1972.
[19] P. Sermpezis, V. Kotronis, P. Gigis, X. Dimitropoulos, D. Cicalese, A. King, and A. Dainotti, "ARTEMIS: Neutralizing BGP hijacking within a minute," IEEE/ACM Transactions on Networking, vol. 26, no. 6, pp. 2471–2486, 2018.
[20] X. Chen et al., "SketchPipe: Toward accurate sketch-based network measurement on multi-pipeline switches with splitless sketch placement," in Proc. 23rd USENIX Symp. Networked Systems Design and Implementation (NSDI), Renton, WA, USA, 2026, pp. 899–914.
[21] A. Cornacchia, T. A. Benson, M. Bilal, and M. Canini, "Observability is eating your cores: Fine-grained analysis of microservice metrics with IPU-hosted sketches," in Proc. 23rd USENIX Symp. Networked Systems Design and Implementation (NSDI), Renton, WA, USA, 2026, pp. 153–174.
[22] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927.
[23] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. Chapman & Hall, 1993.
[24] Z. Liu, A. Manousis, G. Vorsanger, V. Sekar, and V. Braverman, "One sketch to rule them all: Rethinking network flow monitoring with UnivMon," in Proc. ACM SIGCOMM, 2016, pp. 101–114.
[25] Q. Huang, X. Jin, P. P. C. Lee, R. Li, L. Tang, Y.-C. Chen, and G. Zhang, "SketchVisor: Robust network measurement for software packet processing," in Proc. ACM SIGCOMM, 2017, pp. 113–126.
[26] Z. Liu, R. Ben-Basat, G. Einziger, Y. Kassner, V. Braverman, R. Friedman, and V. Sekar, "NitroSketch: Robust and general sketch-based monitoring in software switches," in Proc. ACM SIGCOMM, 2019, pp. 334–350.
[27] Z. Cheng, M. Apostolaki, Z. Liu, and V. Sekar, "TrustSketch: Trustworthy sketch-based telemetry on cloud hosts," in Proc. Network and Distributed System Security Symp. (NDSS), 2024.
[28] P. Chen, Y. Wu, T. Yang, J. Jiang, and Z. Liu, "Precise error estimation for sketch-based flow measurement," in Proc. ACM Internet Measurement Conf. (IMC), 2021, pp. 113–121.
[29] P. Jiang, R. Huang, D. Li, Y. Guo, X. Chen, J. Luan, Y. Ren, and X. Hu, "Auditing frameworks need resource isolation: A systematic study on the super producer threat to system auditing and its mitigation," in Proc. 32nd USENIX Security Symp., 2023, pp. 355–372.
[30] T. F. J.-M. Pasquier, X. Han, M. Goldstein, T. Moyer, D. M. Eyers, M. I. Seltzer, and J. Bacon, "Practical whole-system provenance capture," in Proc. ACM Symp. Cloud Computing (SoCC), 2017, pp. 405–418.
[31] A. Bates, D. Tian, K. R. B. Butler, and T. Moyer, "Trustworthy whole-system provenance for the Linux kernel," in Proc. 24th USENIX Security Symp., 2015, pp. 319–334.
[32] Y. Li, R. Miao, C. Kim, and M. Yu, "FlowRadar: A better NetFlow for data centers," in Proc. 13th USENIX Symp. Networked Systems Design and Implementation (NSDI), 2016, pp. 311–324.
[33] B. Arzani, S. Ciraci, L. F. O. Chamon, Y. Zhu, H. Liu, J. Padhye, B. T. Loo, and G. Outhred, "007: Democratically finding the cause of packet drops," in Proc. 15th USENIX Symp. Networked Systems Design and Implementation (NSDI), 2018, pp. 419–435.